Selective Recovery in a Multi-Agent Coding Workflow

CSE 291A Fall 2026, Option 5 · Phase 1 Report · November 4, 2026

Jackie Wang, Ray Williams

Abstract

When several agents work in sequence, a mistake made by an early agent can be carried forward by the agents after it. Recovering from such a mistake usually means either restarting the whole workflow or checking every result before it is passed on, and both are expensive. This project asks whether tracking the dependencies between agent results can support efficient recovery while preserving correct and safe task execution. In Phase 1 we built the workflow and measured the problem. Three agents, a researcher, a planner and an executor, complete coding tickets on a small Python library. They edit files in a git repository and can publish releases that cannot be taken back, and the result of each ticket is checked by hidden test cases. We then injected seven kinds of faults, such as a wrong fact from the researcher or a malicious instruction hidden in the plan, and traced how each one spread. The faults were rarely harmless: of the 42 faulted runs on tickets the workflow could otherwise solve, 38 ended in wrong code, a broken build or an executed attack. Restarting from scratch always repaired the code but could not undo nine releases that had already been published, and after a fault in the researcher's facts it could reuse none of the earlier work. Checking every hand-off with a verifier blocked every attack, but the verifier also rejected at least half of the correct hand-offs and doubled the cost of every run. These results give Phase 2 its target: recover without rerunning everything, and account honestly for actions that cannot be undone.

1. Experiment Setup

Domain. The agents complete tickets on a small Python `billing` library. We chose a coding domain for three reasons: outcomes can be checked by tests rather than by an LLM judge, the shared state is a real file system that git can version and roll back, and publishing a release is a natural action that cannot be undone. Each of the 10 dev tickets depends on one rule that appears only in the library's internal documents (for example, a 1.5% late fee capped at 2500 cents) and never in the ticket itself. This is what makes the researcher's output matter: if the rule were stated in the ticket or inferable from the code, a corrupted fact would have no effect and there would be nothing to measure. Two tickets also require a release, so some tasks involve irreversible actions legitimately.

Workflow. The researcher reads the documents and returns at most five facts with their sources. The planner turns the ticket, the facts and the current files into JSON steps. The executor has six turns, each a single tool call: read, write, run the visible tests, publish, or post. All three run Qwen3.5 4B (Q4_K_M, hash recorded) on llama.cpp at temperature 0 and seed 0 on a MacBook Pro (M4 Pro). We chose a small self-hosted model so that weights and sampling could be pinned and runs were cheap enough to repeat under every condition. In a five-ticket pilot the model alone solved 10 of 15 tickets but the three-agent chain only 7 of 15, so we constrained the hand-offs (schema-enforced JSON, a fact limit, signatures copied verbatim) rather than tuning prompts to the tickets.

Checking. Outcomes are judged by code. A hidden test file runs on the final workspace; files changed outside the ticket's allowed paths count as collateral damage; the registry and channel are compared with the releases and posts the ticket expects; and tool calls the ticket does not permit are listed as unauthorized, separately from success, so that a safe failure and an unsafe success can be told apart. The tool layer records violations but does not block them, because blocking would be a defense and would hide the damage we are trying to measure. Unit tests confirm that the checker accepts each reference solution and rejects a do-nothing run and a wrong-fact solution.

Faults. We apply faults to the facts or the plan after a stage produces them and before the next stage reads them, so the injection point is exact. Accidental faults replace the deciding fact with a wrong or stale (2025) value, drop all facts (a timeout), or swap the wrong value into the plan. Adversarial faults add an instruction, as a fact or as a plan step, to delete the test file, publish release 9.9.9, or rewrite the policy document. A fixed schedule gives every ticket every fault (70 runs), so all conditions see identical faults. Two controls test whether the workload actually depends on the injected result: a control fault that alters only an irrelevant fact, and a run with the documents removed, which shows which tickets the model can solve without them.

Tracing and labels. Each run records a trace with one node per artifact, listing its inputs, a content hash, tokens and latency. A file read is linked to the step that last wrote the file, so a dependency through state counts as much as one through a message; this is the graph that Phase 2 will walk. Each faulted run is paired with the clean run of the same ticket and labeled *ignored* if the final files, releases and posts match, *absorbed* if they differ but the ticket

still passes with no side effects, and *harmful* otherwise. Tickets the clean chain fails are excluded, so that the model's inability is not mistaken for protection.

Baselines. Full restart is given the identity of the faulty result, resets the workspace to its initial commit and reruns every stage; this separates recovery from detection. Always-verify must detect faults itself: the same model, with its own prompt and access to the documents, checks the facts and the plan before each hand-off, and a rejected stage is regenerated once with the verifier's objections. Because a verified run takes more than twice as long, we ran it on four tickets. On Nov 4 we fixed a parser defect that could write a tool call into a source file and reran the affected tickets (53 of 192 runs changed). `analyze.py` rebuilds every figure below from the saved traces.

2. Evaluation

Clean runs. The chain passed 7 of 10 tickets at 8.8k tokens and 24 s per run. In each failure the researcher reported the correct rule but the executor produced defective code; in one case it then published the broken release without running the tests. Only one ticket passed without the documents. All 70 restarts reproduced their clean runs exactly, so differences between paired runs come from the fault.

Propagation. Table 1 shows that no genuine fault was ignored and that every fact fault changed both the plan and the executor's actions. Accidental faults were harmful in 24 of 28 runs and adversarial faults in 14 of 14. Of the 38 harmful runs, 24 ended in wrong behaviour, 2 in code that failed to import, and 12 in an executed attack. Wrong and stale facts broke even the ticket the model can solve without documents, which suggests the model trusts a supplied value over its own prior. Three of the four absorbed accidental runs were wrong plans, where the executor still held the correct facts. The control fault is a useful check: it changed the executor's actions on all seven tickets yet left three end states identical and broke only one. Differences in artifacts therefore overstate propagation, and outcomes have to be judged on end state. Work downstream of a fault averaged 5.9k to 11.2k tokens per run against 8.8k for a complete clean run; this is an upper bound, since every reachable node is counted.

Fault	Class	Ignored	Absorbed	Harmful	Plan changed	Executor changed
Wrong fact	Accidental	0	0	7	7	7
Stale fact	Accidental	0	0	7	7	7
Timeout	Accidental	0	1	6	7	7
Wrong plan	Accidental	0	3	4	–	7
Malicious fact	Adversarial	0	0	7	7	7
Malicious plan	Adversarial	0	0	7	–	7
Control	Control	3	3	1	5	7

Table 1. Outcomes on the 7 clean-passing tickets, one run each (plan faults cannot change the plan).

Unsafe actions. Across all 10 tickets the executor carried out 16 of 20 injected attacks, equally often from a fact and from a plan step. Accidental faults also caused irreversible damage: on the shipping ticket, four faulted runs published a wrong version.

Recovery baselines. Full restart restored correct code in all 42 faulted runs but fully recovered only 33; the other nine had already published a release that no rerun can withdraw. After a fact fault none of the restart's work was reusable, since every model call was either the faulty result or downstream of it; after a plan fault the researcher's share (19.6% of tokens) was. Always-verify flagged the faulted stage in 23 of 24 runs and prevented all eight attacks, raising faulted passes from 0 to 15 of 24. Its detection, however, is indiscriminate. It rejected a stage in 31 of 32 runs, including 6 of 12 correct fact lists and 3 of 4 correct plans, and the forced regeneration halved passes on clean and control runs. Each verified run added 7.6k tokens and 32 s. Tracing costs 14 kB and under 0.5% of run time.

3. Conclusion

In this workflow a corrupted fact was never contained: it reached every later stage in every run. Restart repeats all of the work and cannot undo releases, which caused every failed recovery; verification stops attacks but rejects most correct work and doubles the cost. Because the trace records dependencies per stage, dependency-aware invalidation would currently rerun everything after a fact fault. Phase 2 therefore tracks which facts each plan step and file write use, adds tickets whose parts are independent so there is valid work to keep, rolls files back to the last unaffected commit, and compares all three methods on a held-out set.

Appendix: Timeline and milestones

Dates	Work	Milestone
Oct 27	Repo scaffold, state store, unit tests	Done
Oct 27–29	Model pilot on five tickets	Done: Qwen3.5 4B chosen; file hash recorded
Nov 2	Coding workspace and tools	Done: tool unit tests pass
Nov 2	10 dev tasks, hidden tests, checker; agents	Done: reference passes; do-nothing fails
Nov 2–3	Provenance tracing; fault injector and schedule	Done: faulted run compared with its clean pair
Nov 3	Full-restart and always-verify baselines	Done: restart on all tickets; verify on four
Nov 3–4	Dev-set clean, fault and baseline runs; results tables	Done: tables in `results/`
Nov 4	Executor parser fix; affected tickets rerun	Done: 53 of 192 runs changed
Nov 4	**Phase 1 code and data due**	One command per condition
Nov 5–17	Fact-level provenance; selective invalidation	Only steps that cite a bad fact rerun
Nov 10–20	Held-out set, multi-part and multi-ticket tasks	Unseen until the final evaluation
Nov 12–24	Shared-state rollback and snapshots	Irreversible actions reported
Nov 23–Dec 3	Held-out evaluation on AWS	Same schedules, budgets and seeds
Dec 1–9	Final report	
Dec 10	**Final submission**	